

Part 5

Parallel Hardware and Software

A supercomputer is a device for turning
compute-bound problems into I/O-bound
problems

Ken Batchner

It's perfectly feasible for specialists in disciplines other than computer science and computer engineering to write parallel programs. However, in order to write efficient parallel programs, we do need some knowledge of the underlying hardware and system software. It's also very useful to have some knowledge of different types of parallel software, so in this chapter we'll take a brief look at a few topics in hardware and software. We'll also take a brief look at evaluating program performance and a method for developing parallel programs.

5.1 Background

Parallel hardware and software have grown out of conventional serial hardware and software: hardware and software that runs (more or less) a single job at a time. So in order to better understand the current state of parallel systems, let's begin with a brief look at a few aspects of serial systems.

5.1.1 The von Neumann architecture

The "classical" von Neumann architecture consists of main memory, a central processing unit (CPU) or processor or core, and an interconnection between the memory and the CPU. Main memory consists of a collection of locations, each of which is capable of storing both instructions and data. Every location consists of an address, which is used to access the location and the contents of the location—the instructions or data stored in the location.

The central processing unit is divided into a control unit and an arithmetic and logic unit (ALU). The control unit is responsible for deciding which instructions in a program should be executed, and the

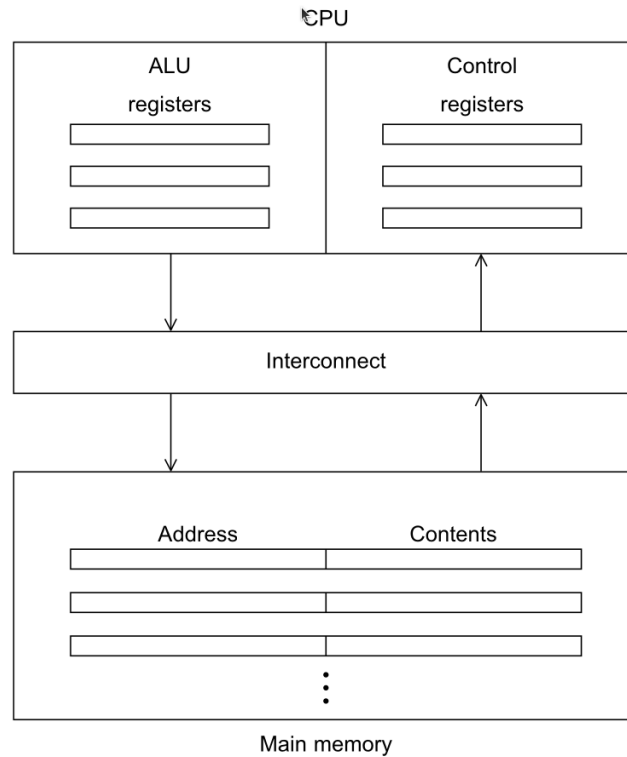


Figure 5.1: The von Neumann architecture.

ALU is responsible for executing the actual instructions. Data in the CPU and information about the state of an executing program are stored in special, very fast storage called **registers**. The control unit has a special register called the **program counter**. It stores the address of the next instruction to be executed.

Instructions and data are transferred between the CPU and memory via the interconnect. This has traditionally been a bus, which consists of a collection of parallel wires and some hardware controlling access to the wires. A von Neumann machine executes a single instruction at a time, and each instruction operates on only a few pieces of data. See Figure 5.1.

When data or instructions are transferred from memory to the CPU, we sometimes say the data or instructions are **fetch**ed or **read** from memory. When data are transferred from the CPU to memory, we sometimes say the data are **written** to memory or **stored**. The separation of memory and CPU is often called the **von Neumann bottleneck**, since the interconnect determines the rate at which instructions and data can be accessed. The potentially vast quantity of data and instructions needed to run a program is effectively isolated from the CPU. In 2020 CPUs are capable of executing instructions more than one hundred times faster than they can fetch items from main memory.

5.1.2 Modifications to the von Neumann Model

As we noted earlier, since the first electronic digital computers were developed back in the 1940s, computer scientists and computer engineers have made many improvements to the basic von Neumann architecture. Many are targeted at reducing the problem of the von Neumann bottleneck, but many are

also targeted at simply making CPUs faster. In this section we'll look at three of these improvements: caching, virtual memory, and low-level parallelism.

Caching

Caching is a technique used to overcome the von Neumann bottleneck by reducing the delay between the CPU and main memory. It works by storing frequently used data and instructions in a small, fast memory (cache) that is closer to the CPU. This reduces the need for repeated data transfers over the slower connection to main memory. In essence, caching both widens the data pathway (by fetching blocks of data instead of single items) and brings memory closer to the CPU. A CPU cache is typically either built directly on the processor chip or placed nearby, allowing much faster access than main memory.

Once a cache is introduced, the main challenge is deciding which data and instructions to store in it. The standard approach relies on the principle of locality — the observation that programs tend to reuse data and instructions that are close in memory to those recently accessed.

- Spatial locality: After accessing one memory location, a program often accesses nearby locations (e.g., elements of an array).
- Temporal locality: Recently used data or instructions are likely to be used again soon.

For example, consider the following loop:

```
1  for (i = 0; i < 1000; i++)  
2      sum += z[i];
```

The array elements $z[0], z[1], \dots, z[999]$ are stored in **contiguous memory locations**. Hence, accessing $z[i]$ is almost always followed by accessing $z[i+1]$, making caching these nearby values highly efficient.

The principle that accessing one memory location is often followed by accessing a nearby location is known as the **principle of locality**. After accessing a specific memory location (instruction or data), a program typically accesses a nearby location (**spatial locality**) and is likely to reuse it again in the near future (**temporal locality**).

To exploit this behavior, the system uses an **effectively wider interconnect** to transfer data and instructions. Instead of operating on individual memory items, each memory access retrieves an entire **block** of consecutive data or instructions. These blocks are known as **cache blocks** or **cache lines**. A typical cache line stores about 8 to 16 times more information than a single memory location. In our example, if a cache line stores 16 floats, then when we first go to add $\text{sum} += z[0]$, the system might read the first 16 elements of z , $z[0], z[1], \dots, z[15]$ from memory into cache. So the next 15 additions will use elements of z that are already in the cache.

Conceptually, a CPU cache can be viewed as a single, unified structure. In practice, however, it is organized into multiple **levels of hierarchy**. The **level 1 (L1) cache** is the smallest and fastest,

while higher levels such as **L2** and **L3** are progressively larger and slower. Most modern systems contain at least two cache levels, and three levels are quite common.

Caches typically store **copies of data** from slower memory. In multilevel cache systems, a lower-level cache (faster and smaller) can be viewed as a cache for a higher-level one. For example, data stored in an L1 cache often also exists in the L2 cache. However, in some systems, caches are designed to avoid duplication, so data in L1 may not appear in L2 but will still be stored in main memory.

When the CPU needs an instruction or data, it searches the cache hierarchy in order: first L1, then L2, and so on, before accessing main memory if necessary. If the requested information is found in a cache, the event is called a **cache hit**; if not, it is a **cache miss**. These events are often described by level, for instance, an **L1 miss** followed by an **L2 hit**.

The terms **read** and **write** also apply to cache operations. For example, a CPU may read an instruction from the L2 cache or write data to the L1 cache.

When the CPU attempts to read data or instructions and a **cache read miss** occurs, the processor retrieves the corresponding **cache line** from main memory and stores it in the cache. This process can temporarily **stall the CPU**, as it must wait for the slower memory to deliver the requested data. For instance, when reading $z[0]$, the processor may pause while the cache line containing $z[0]$ is fetched from memory into the cache.

When the CPU writes data to the cache, the values in the cache and main memory may become **inconsistent**. Two common strategies handle this:

- **Write-through cache:** Every time data is written to the cache, it is simultaneously written to main memory, ensuring consistency at all times.
- **Write-back cache:** Data written to the cache is not immediately updated in main memory. Instead, the modified cache line is marked as **dirty**, and when that line is replaced, the updated data is then written back to memory.

It is important to note that the behavior of the CPU cache is **controlled by the system hardware**, and programmers cannot directly choose which data or instructions are stored in it. However, understanding the principles of **spatial** and **temporal locality** enables programmers to influence cache efficiency indirectly through the way they access memory.

For example, the C programming language stores two-dimensional arrays in **row-major order**. Although a two-dimensional array is conceptually a rectangular grid, memory itself is organized as a one-dimensional sequence. In row-major storage, the elements of the first row are stored first, followed by those of the second row, and so on.

Consider the following two code fragments:

```
1  for (i = 0; i < N; i++)
2      for (j = 0; j < N; j++)
3          sum += A[i][j];    // Row-major access
```

```
1  for (j = 0; j < N; j++)
2      for (i = 0; i < N; i++)
3          sum += A[i][j];    // Column-major access
```

The first version accesses elements in contiguous memory locations (row by row), allowing the cache to exploit **spatial locality** effectively. The second version accesses memory in non-contiguous order (column by column), resulting in poorer cache performance.

Virtual Memory

Caches allow the CPU to **quickly access instructions and data** stored in main memory. However, when executing very large programs or handling large data sets, it may be impossible to keep all necessary information in main memory at once. This limitation becomes even more significant in **multitasking operating systems**, where multiple programs share memory resources.

To maintain the illusion of simultaneous execution, the operating system ensures that the instructions and data needed for the next program's **time slice** are loaded into main memory. Even with large memory capacity, **multiple programs must share available memory**, and this sharing must be managed carefully to prevent one program's data and instructions from being corrupted by another.

Virtual memory was developed to allow main memory to act as a **cache for secondary storage**. It takes advantage of the **principles of spatial and temporal locality** by keeping only the **active portions** of running programs in main memory, while inactive parts are stored in a region of secondary storage known as **swap space**. Similar to CPU caches, virtual memory transfers data and instructions in **blocks**, called **pages**. Because accessing secondary storage is significantly slower—often hundreds of thousands of times slower than main memory—pages are relatively large, typically ranging from **4 to 16 kilobytes** in modern systems.

Problems arise if **physical memory addresses** are assigned to program pages at compile time, since each page would then be tied to a specific memory block. In a **multitasking operating system**, multiple programs could attempt to use the same memory regions, leading to conflicts.

To prevent this, when a program is compiled, its pages are given **virtual page numbers** rather than fixed physical addresses. During execution, the operating system creates a **page table** that maps these virtual page numbers to **physical memory addresses**. Whenever the program accesses a virtual address, the page table translates it into the corresponding physical address. By managing page tables, the operating system ensures that the memory spaces of different programs remain **isolated and protected** from one another.

Instruction Level Parallelism

Instruction-level parallelism (ILP) aims to improve processor performance by enabling **multiple functional units** within the CPU to execute instructions simultaneously. There are two primary approaches to achieving ILP:

- **Pipelining:** Functional units are organized into sequential stages, allowing different stages of multiple instructions to be processed concurrently.

As an example involving computation, suppose we want to add the floating point numbers 9.87×10^4 and 6.54×10^3 . Then we can use the following steps: Now if each of the operations

Time	Operation	Operand 1	Operand 2	Result
(-)	Fetch operands	9.87×10^4	6.54×10^3	
1	Compare exponents	9.87×10^4	6.54×10^3	
2	Shift one operand	9.87×10^4	0.654×10^4	
3	Add	9.87×10^4	0.654×10^4	10.524×10^4
4	Normalize result	9.87×10^4	0.654×10^4	1.0524×10^5
5	Round result	9.87×10^4	0.654×10^4	1.05×10^5
6	Store result	9.87×10^4	0.654×10^4	1.05×10^5

takes one nanosecond (10^{-9} seconds), the addition operation will take seven nanoseconds. So if we execute the code

```

1  float x[1000], y[1000], z[1000];
2  for (i = 0; i < 1000; i++)
3      z[i] = x[i] + y[i];

```

the for loop will take something like 7000 nanoseconds.

As an alternative approach, consider dividing a floating-point adder into seven functional units, each responsible for a specific operation — for example, fetching operands, comparing exponents, adding values, and normalizing results. Each unit's output serves as the input to the next stage.

Although a single floating-point addition would still take seven nanoseconds, the stages can be overlapped through pipelining. For instance, while one pair of operands ($x[0]$, $y[0]$) is having its exponents compared, the next pair ($x[1]$, $y[1]$) can already be fetched. In general, this allows seven different stages of seven different additions to be executed simultaneously.

After the initial setup time, the pipelined loop produces one result per nanosecond instead of one every seven nanoseconds. Consequently, the total time for executing the loop decreases from 7000 nanoseconds to 1006 nanoseconds, yielding an almost sevenfold performance improvement.

- **Multiple issue:** The processor can initiate several instructions at once, enabling parallel execution within a single clock cycle.

Pipelines improve performance by taking individual pieces of hardware or functional units and connecting them in sequence. Multiple issue processors replicate functional units and try to simultaneously execute different instructions in a program. For example, if we have two complete floating point adders, we can approximately halve the time it takes to execute the loop

```

1  for(i=0 ; i<1000 ; i++)
2      z[i]=x[i]+y[i] ;

```

While the first adder is computing $z[0]$, the second can compute $z[1]$; while the first is computing $z[2]$, the second can compute $z[3]$; and so on.

Both techniques are fundamental features of *modern CPU architectures*.

Hardware Multithreading

Instruction-level parallelism (ILP) can be challenging to exploit, especially in programs with long sequences of dependent instructions, since each operation relies on the result of the previous one. For example, in the direct computation of Fibonacci numbers:

```
1  f[0] = f[1] = 1;
2  for (i = 2; i <= n; i++)
3      f[i] = f[i-1] + f[i-2];
```

each iteration depends on the results of the two preceding iterations, leaving little or no opportunity for simultaneous instruction execution.

Thread-level parallelism (TLP), on the other hand, achieves parallelism by executing multiple threads concurrently. This provides a coarser-grained form of parallelism than ILP, since the parallel units (threads) encompass larger sections of code compared to individual instructions.

Hardware multithreading enables a processor to continue performing useful work when the currently executing task is stalled, such as when waiting for data to be fetched from memory. Instead of searching for parallelism within a single thread, the processor can switch to executing another thread that is ready to run.

For this approach to be effective, the system must allow for very fast thread switching. In older systems, threads were often implemented as separate processes, and switching between them could take as long as executing thousands of instructions, making such multithreading inefficient. Modern hardware multithreading, however, minimizes this switching overhead, allowing multiple threads to share processor resources efficiently.

In **fine-grained multithreading**, the processor switches between threads after executing each instruction, skipping any threads that are stalled. This approach effectively reduces wasted processor time caused by stalls. However, it also means that a thread capable of executing a long sequence of instructions must wait between each instruction, potentially lowering performance.

Coarse-grained multithreading addresses this issue by switching threads only when the currently executing thread is stalled on a time-consuming operation (such as a memory load from main memory). This reduces the need for instantaneous thread switching and allows more efficient execution of long instruction sequences. Nevertheless, the processor may remain idle during shorter stalls, and switching threads still introduces some delay.

5.1.3 Parallel Hardware

Multiple issue and pipelining can clearly be considered to be parallel hardware, since functional units are replicated. However, since this form of parallelism isn't usually visible to the programmer, we're treating both of them as extensions to the basic von Neumann model, and for our purposes, parallel hardware will be limited to hardware that's visible to the programmer.

SIMD Systems

In parallel computing, *Flynn's taxonomy* [Flynn1966] is a widely used framework for classifying computer architectures based on the number of instruction streams and data streams a system can process simultaneously.

A traditional von Neumann system is categorized as a single instruction stream, single data stream (SISD) system, because it executes one instruction at a time and can fetch or store only one data item at a time.

Single Instruction, Multiple Data (SIMD) systems achieve **data-level parallelism** by applying the same instruction to multiple data elements simultaneously. A single control unit broadcasts instructions to multiple ALUs, each operating on its local data or remaining idle if no data is available.

For example, in vector addition of arrays x and y , an SIMD system with n ALUs can add all elements in parallel. If the system has fewer ALUs than elements, the computation is performed in blocks, and some ALUs may remain idle in the final block. SIMD provides efficient parallel execution for operations on large data sets with the same instruction applied repeatedly.

The requirement that all ALUs in a SIMD system execute the same instruction, or remain idle, can reduce overall performance. For example, consider the conditional vector addition:

```
1  for (i = 0; i < n; i++)  
2      if (y[i] > 0.0) x[i] += y[i];
```

Each ALU must check whether its corresponding $y[i]$ is positive. If $y[i]$ is not positive, that ALU remains idle while others perform the addition, leading to underutilization.

Classical SIMD systems also require synchronous execution, where all ALUs wait for the next instruction broadcast, and ALUs do not have local instruction storage to delay execution. SIMD systems are ideal for **data-parallel tasks**, where simple loops operate on large arrays and each processor applies the same instruction to different subsets of data. While highly efficient for such problems, SIMD systems may perform poorly on tasks that do not fit this data-parallel model.

Vector Processors

Modern CPUs often include vector processing capabilities, which extend SIMD principles:

- Vector registers: Store multiple operands (typically 4–128 elements) for simultaneous operations.
- Vectorized and pipelined units: Apply the same operation to each element (or pair of elements), enabling SIMD execution.
- Vector instructions: Operate on entire vectors at once. For example, a loop like

```
1  for (i = 0; i < n; i++)  
2      x[i] += y[i];
```

requires only one load, add, and store per vector block, instead of per element.

- Interleaved memory: Multiple memory banks allow concurrent access to successive elements, reducing delays.
- Strided access and scatter/gather: Special hardware efficiently handles vectors with regular (strided) or irregular (scatter/gather) memory patterns.

Vector processors offer high performance and ease of use for many applications. Modern vectorizing compilers can automatically identify code that can be vectorized and provide feedback on loops that cannot, enabling programmers to optimize performance. Key advantages include:

- High memory bandwidth: Every loaded data element is typically used, unlike cache-based systems.
- Efficient vectorization: Loops with regular operations can be executed very quickly.

Limitations include:

- Poor performance on irregular data structures.
- Limited scalability: There is a practical limit to vector length, making it difficult to extend to arbitrarily large problems.

Graphics Processing Units (GPUs)

GPUs are designed to accelerate real-time graphics by processing points, lines, and triangles to generate pixels for display. They use a graphics processing pipeline, with several stages programmable via short shader functions. Shader functions are inherently parallel, as they can be applied simultaneously to multiple vertices or pixels.

Modern GPUs exploit **SIMD** parallelism within each core, which is achieved by including a large number of ALUs (e.g., 80 per core). This allows many elements to be processed concurrently with similar control flow.

Processing graphics involves handling large amounts of data—hundreds of megabytes per image are common. To maintain high throughput and avoid memory stalls, GPUs rely heavily on hardware multithreading, storing the state of many suspended threads simultaneously. The number of threads depends on the resources required by the shader function. However, GPUs perform less efficiently on small problems due to underutilized ALUs.

Importantly, GPUs are not purely SIMD. While each core executes SIMD operations, modern GPUs have multiple cores capable of executing independent instruction streams, providing a combination of data- and thread-level parallelism.

5.1.4 Shared- and Distributed- Memory Systems

In shared-memory systems with multiple multicore processors, the memory interconnect can be organized in two ways:

- **Uniform Memory Access (UMA):** All processors are directly connected to all of main memory. Access time to any memory location is uniform across all cores, simplifying programming.
- **Nonuniform Memory Access (NUMA):** Each processor has a direct connection to a portion of main memory. Access to local memory is faster than access to memory connected to other processors. NUMA systems can support larger memory capacities and offer faster local access, but require awareness of memory placement for optimal performance.

UMA systems are generally easier to program due to uniform access times, whereas NUMA systems trade programming simplicity for higher performance and scalability.

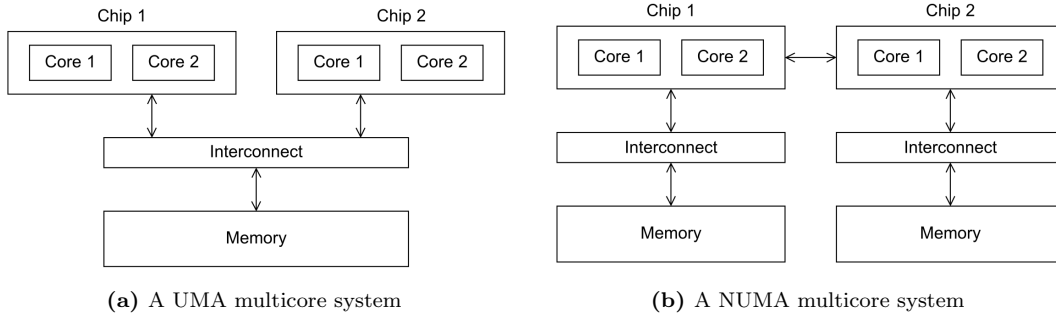


Figure 5.2: Shared-memory systems

The most common form of distributed-memory system is the cluster. A cluster consists of multiple commodity computers (e.g., PCs) connected through a commodity interconnection network, such as Ethernet. Each computational unit, or node, typically contains one or more multicore processors and functions as a shared-memory system. Since clusters combine shared-memory nodes connected via

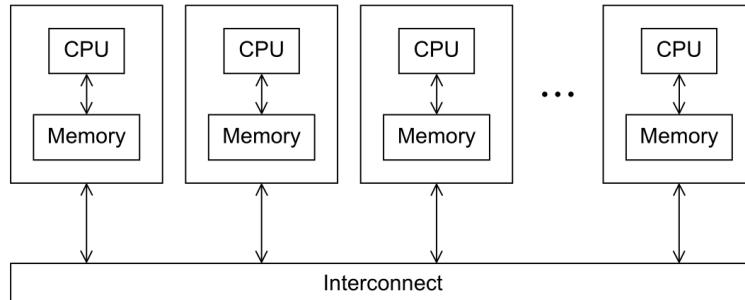


Figure 5.3: A distributed-memory system

message passing, they are often referred to as hybrid systems. Today, it is generally implied that a cluster will have shared-memory nodes interconnected through a distributed-memory network.

A grid extends this concept by providing an infrastructure that connects large networks of geographically distributed computers into a unified distributed-memory system. Unlike clusters, grids are typically heterogeneous, meaning that their nodes may differ in hardware architecture and performance characteristics.

Latency and Bandwidth

Whenever data is transferred—from main memory to cache, cache to register, disk to memory, or between nodes in a distributed-memory or hybrid system—it is important to understand how long the transmission will take. Two key parameters characterize the performance of any interconnection network:

- **Latency (l):** the delay between the moment the source begins transmitting and the moment the first byte reaches the destination.
- **Bandwidth (b):** the steady rate at which data continues to arrive once transmission has begun, typically expressed in bytes per second.

The total time required to send a message of size n bytes can therefore be approximated by

$$T_{\text{message}} = l + \frac{n}{b},$$

where the first term represents the initial startup delay, and the second term accounts for the time needed to transfer the entire message once transmission is underway.

5.2 High Performance Computing

In this lecture we give an overview of High Performance Computing (HPC). Here are some definitions of HPC gathered using google:

From [InsideHPC.com](#):

“HPC most generally refers to the practice of aggregating computing power in a way that delivers much higher performance than one could get out of a typical desktop computer or workstation in order to solve large problems in science, engineering, or business.”

From [NetAPP.com](#):

“HPC is the ability to process data and perform complex calculations at high speeds. To put it into perspective, a laptop or desktop with a 3 GHz processor can perform around 3 billion calculations per second. While that is much faster than any human can achieve, it pales in comparison to HPC solutions that can perform quadrillions of calculations per second.”

The [Wikipedia HPC page](#) includes a lot of information but does not actually define HPC.

From [Techpedia.com](#):

“HPC is the use of super computers and parallel processing techniques for solving complex computational problems. HPC technology focuses on developing parallel processing algorithms and systems by incorporating both administration and parallel computational techniques.

High-performance computing is typically used for solving advanced problems and performing research activities through computer modeling, simulation and analysis. HPC systems have the ability to deliver sustained performance through the concurrent use of computing resources.”

From [National Insititute for Comptutational Science @U. Tennessee, Knoxville](#):

“HPC, is the application of “supercomputers” to computational problems that are either too large for standard computers or would take too long.”

A recurrent theme in these descriptions of HPC is using parallel computing techniques on large scale systems (like clusters or super-computers) to exceed the performance and/or capacity of a typical laptop or desktop computer. In reality there is more to HPC than merely running large calculations on large scale clusters. In the following sections we break down different aspects of HPC.

5.2.1 Calculations with large data requirements

When you purchased your laptop and chose the model with 8GB memory you probably didn’t think about the intrinsic limitations of that choice. With 8GB of random access memory (RAM) you can keep at most one billion double precision variables in system memory at one time ¹

What do you do if you want to work on problems that need more than a billion doubles, for instance when you are solving a problem with tens of billions of degrees of freedom? You may be able to install extra RAM, or buy a computer with more RAM at some considerable cost of course. Indeed some workstations are equipped with a terabyte of RAM, raising the capacity to a hundred billion doubles simultaneously in memory. Unfortunately, the time it takes to process a problem typically increases with the problem size, and with that extra memory, we would be wise to upgrade the central processing unit (CPU). But what should we do if our target problem requires many terabytes of data?

5.2.2 Calculations that take a long time

While some calculations may require large amounts of data, other calculations may conversely have very small memory footprints yet take an inordinately long time²

Consider for instance the task of finding all the prime numbers up to a given large number N . One brute force approach with almost no memory requirements is to test if each number is divisible by all the integers less than its square root. Admittedly this is a very inefficient way of finding the primes but let’s suppose for the sake of this example that in some parallel universe [Eratosthenes](#) did not invent the prime sieve and mathematicians had paid no attention to primes and thus brute force calculation is the only known strategy. In that universe we would be obliged to perform (pessimistically) $\mathcal{O}(N^{3/2})$ divisions to find the primes up to N . Now let’s imagine that in this alternative universe, although the Mathematicians had been asleep at the wheel, computer engineering was just as advanced. The alt world computer scientists would observe that each integer can be tested independently for primality independently of all the other integers. Thus if we split the primality tests amongst P processors then

¹You can conceivably perform calculations with more than a billion doubles on your laptop if you are patient and are ok using the slower hard drive or solid-state drive usually used for storing files to store your data during the calculation.

²The word inordinately derives from the Latin root verb “ordinare” which means to arrange. Entertainingly this root verb is shared by the French word for computer “ordinateur”. Furthermore, the French word “ordinateur” [was introduced into the language by IBM in the 1950s](#) and it became a commonly used word before they could trademark it.

it should be possible to reduce the calculation time from $\mathcal{O}(N^{3/2})$ to $\mathcal{O}(N^{3/2}/P)$ which will make it possible to compute the primes to larger N for a fixed wall clock time. However, that alternative universe will still come to an end before the calculations terminate for sufficiently large N .

Fortunately in our universe, the Sieve of Eratosthenes was discovered early and improved over time. [It and more recent variants](#) are significantly more efficient than brute force testing making the above discussion moot. Furthermore there are many [alternative ways for testing for primality](#) if that is your goal.

5.2.3 Moore transistors, Moore compute

In 1965 Gordon Moore of Intel predicted a decade of exponential increase³ in economically viable processor capability improvements.

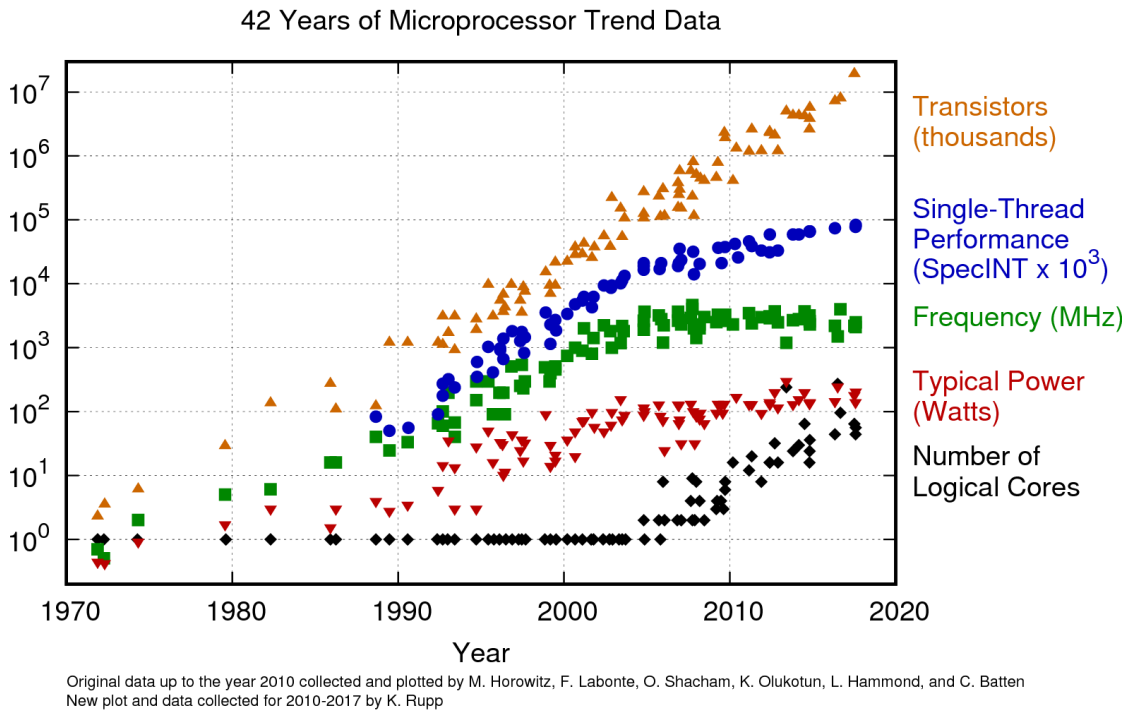


Figure 5.4: Trends in processor design. Image sourced from [Karl Rupp](#). Source data is available on [GitHub here](#).

The increase in processor complexity and efficiency coupled with increased clock frequency continued unabated for forty years (from 1965 to 2005) until thermal considerations finally limited clock frequency for monolithic “serial” processors. In Figure 5.4 we show the growth in single core processing power. Notice the leveling off in performance in 2005. In particular the single thread performance benchmark shows little recent increase and the manufacture spec core frequency has declined slightly over time. The “Typical Power” requirement of the CPU is the key factor that limits the frequency and thread performance. To be precise dissipating the heat generated by the processor at higher clock frequencies is the major hurdle that forces manufacturers to curtail ever increasing the CPU clock

³See the first graph of [1] for an honest to goodness exponential growth chart that actually came to pass. This is one of only a few true exponentially varying processes that has continued for forty years.

frequency. This is because the power consumption of the CPU varies nonlinearly with clock frequency, some analyses show a cubic relationship between clock frequency and power consumption of the CPU !

After 2005 the bulk of CPU performance improvements have resulted from increasing the number of separate processor cores in each CPU. Nowadays the task of exploiting multiple cores on a single CPU rests with the programmer. In the following lectures we will introduce the predominant programming models used to divvy work up amongst multiple cores. We will focus initially in particular on the Open Multi-Processing (OpenMP) approach to parallel programming. Writing an OpenMP program entails annotating a regular Fortran, C, or C++ program with special compiler directives that instruct the compiler to split work amongst multiple cores.

A carefully crafted OpenMP implementation of a calculation that is particularly well suited to execution by multiple threads on multiple cores of a modern CPU can attain over an order of magnitude performance improvements over the same program running in serial on a single CPU core. However the key phrases here are “carefully crafted” and “particularly well suited to execution by multiple thread”. This is by no means true of all calculations and should serve as a reality check before you get too excited.

The brutal truth is that a modern Intel CPU with over two dozen processor cores can deliver less than 5% of its potential performance when running a serial code. In fact a serial code that does not take advantage of the single instruction multiple data (SIMD) vector floating point units (FPU) built into each CPU core and only executes scalar arithmetic operation at time, i.e. one floating point operation per clock cycle, will achieve less than 1% of the theoretical peak of the CPU floating performance. Imagine that you only score 1% in every homework assignment, not a great achievement, but so far in your academic careers it is very likely that any code you have written has achieved that very modest level of achievement.

5.2.4 Scaling up

The US Department of Energy (DOE) is tasked with nuclear stockpile management and general computational modeling for energy applications. Modeling for energy related applications inevitably involves evaluating computational models quite often involving complicated dynamical systems with potentially trillions of state variables. For example modeling turbulence in fuel injectors, heat exchange processes and neutronics in nuclear reactors, efficient energy capture in wind farms, design and analysis of nanoscale manufacturing processes, urban environmental heating, and many more energy related problems. The DOE has been a national and worldwide leader in building increasingly powerful supercomputers and software to deploy on these systems to address these large scale modeling problems.

Recently the DOE started the Exascale Computing Project (ECP) to address many of the challenges that result from the next step in the ramp up from current petascale generation supercomputers to exascale systems. The ECP is responsible for developing software that can deploy on systems that are theoretically capable of 10^{18} floating point operations per second. That is a billion billion operations per second, an inconceivable high rate for performing double precision arithmetic ! The initial exascale generation of systems will primarily deliver this performance by including thousands of extremely powerful graphics processing units (GPU). We will contemplate GPUs later on in this course. For the moment all you need to know is that the most powerful GPUs in the current generation have more than 2000 floating point units organized into 80 cores with each core controlling 32 wide SIMD vector FPUs.

5.2.5 The big competition

The US DOE is a dominant participant in HPC but governments, corporations, universities, and private organizations worldwide also operate supercomputers. The top500.org website maintains a list of the top 500 most powerful systems worldwide. Each system on the list is registered by its organization with the Top500 organization. Thus the list is not complete and some additional powerful systems may be omitted from the list. To be included on the list some standard benchmarks must be run on the system to determine its throughput capability. These benchmarks and basic system capabilities must be submitted together. The Top 500 list is assembled from this self-reported information. In Table 5.1 we show the top 5 supercomputers from this list as of June 2025.

Rank	Site	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
1	DOE/NNSA/LLNL United States	El Capitan, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, HPE	11,039,616	1,742	2,746	29,580
2	DOE/NNSA/LLNL United States	Frontier, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE	8,699,9042	1,194	1,679	22,703
3	DOE/SC/Argonne National Laboratory, United States	Aurora, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11	9,264,128	1,012	1,980	38,698
4	EuroHPC/FZJ	Jupiter Booster, BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200	4,801,344	793	930	13,088
5	Microsoft Azure	Eagle, Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR	2,073,600	561	846	

Table 5.1: Top 5 supercomputers as of June 2025, data from [Top500.org](https://top500.org).

The benchmarks used to rank the computational performance of the systems include the performance of [LINPACK](#) benchmark tests, and this is somewhat controversial because it consists of a set of synthetic benchmarks involving the factorization of large and dense matrices. [The controversy](#) stems from the LINPACK workload not being representative of typical large-scale calculations. Recently the [HPCG benchmark](#) was proposed as a more suitable benchmark than LINPACK for testing supercomputers in that it entails a conjugate gradient solver using a sparse matrix representation. However, HPCG has a relatively low arithmetic intensity (number of floating point operations per memory access) making it less attractive as a way for vendors to highlight the raw computational power of their systems.

5.2.6 Energy matters

The quintessential point of the HPCG benchmark is that most computational tasks follow a similar pattern: read data from memory, perform a small number of arithmetic operations, and then write the result to memory. This type of task exercises the memory system: reading data from system memory, transmitting data by wire, and percolating the data through multiple levels of memory caches before it reaches the CPU. It barely engages the floating point capability of the vector SIMD units. Assuming that there is enough data that it does not fit in fast cache memory then the data has to continually

stream from slow system memory. In this scenario the achieved floating point performance will be substantially low compared to manufacturer peak floating point performance claims, maybe even at [low single digit utilization of the CPU floating point capability](#) !

Table 1. Technology and circuit projections for processor chip components.			
Process technology	2010	2017	
	40 nm	10 nm, high frequency	10 nm, low voltage
V_{DD} (nominal)	0.9 V	0.75 V	0.65 V
Frequency target	1.6 GHz	2.5 GHz	2 GHz
Double-precision fused-multiply add (DFMA) energy	50 picojoules (pJ)	8.7 pJ	6.5 pJ
64-bit read from an 8-Kbyte static RAM (SRAM)	14 pJ	2.4 pJ	1.8 pJ
Wire energy (per transition)	240 femtojoules (fJ)	150 fJ/bit/mm	115 fJ/bit/mm
	per bit per mm		
Wire energy (256 bits, 10 mm)	310 pJ	200 pJ	150 pJ

Figure 5.5: Estimates of energy usage for fused multiply-add operation, accessing data, and moving data on typical processor from 2010 and prediction for processor in 2017. Table source: [2].

Figure 5.5 illustrates why data movement is such a significant issue for modern processors. The energy dissipated when moving data from RAM to processor depends on distance moved and is considerably larger than the energy expended to perform a fused-multiply-add operation (FMADD). Even if the data moves only 10mm between memory and processor the energy dissipated in the wire during the move exceeds the energy dissipated during the FMADD.

The HPCG benchmark exposes the quandry of modern computing because it requires a very small number of arithmetic operations performed for each piece of data moved. Thus performance for HPCG is much lower than LINPACK since the latter performs a very large number of arithmetic operations in dense linear algebra block operations. Yet somehow the top500.org judges supercomputers by their raw floating point performance not on the strength of their memory systems. Are we building the wrong systems targeting dense linear algebra for huge linear systems that nobody will ever need as measured with the LINPACK benchmark ? The new exascale leadership systems each cost in excess of a half billion USD and they will inevitably be judged in part by their LINPACK performance. Digest that for a moment.

5.2.7 Less is Moore

In the last section we observed that reading data from system memory consumes a picojoule per bit per 10 mm moved over wire. A picojoule is $10^{-12}J$, a miniscule fraction of a Joule⁴ However, a large scale

⁴Definition: One joule is the equivalent of one watt of power radiated or dissipated for one second. [Example from wiki](#): “The energy required to lift a medium-sized tomato up 1 metre (3 ft 3 in) (assume the tomato has a mass of approximately 100 grams (3.5 oz)).”

system running at full capacity will move vast amounts of data. The energy used to move data and to perform calculations all adds up. For instance, the Summit supercomputer at Oak Ridge National Lab [consumes up to 13MW](#)(mega-Watts) of power. For reference 1MW can power approximately 650 homes in the US (see [for a discussion](#)).

The power consumption of a large scale computing system has four important ramifications:

1. **Cost:** 1MW-year of power costs approximately \$1,000,000 per year (estimate obtained from this [web-site](#)).
2. **Environmental impact:** the power for a supercomputer must be supplied on a continuous and reliable basis. Renewable resources may be an option, but might not meet stringent quality and reliability requirements.
3. **Power supply:** the DOE leadership facilities are typically collocated with power stations. The maximum reliable load of the power station dictates the power budget for the supercomputer.
4. **Cooling:** the bi-product of computing is heat, lots of heat. The excess heat needs to be removed requiring considerable cooling systems. Some facilities use excess heat for other purposes.

The [Green Top 500 Supercomputer list](#) was established to address the issue of improving environmentally sensitive computing. Systems on the Green Top500 list are ranked by metrics like GFLOPS/Watt, i.e. by their computational efficiency.

According to [the Green Top500 website](#)

“Historically, the Green500 started back in April 2005 – after a keynote talk by Dr. Wu-chun Feng⁵ at the IEEE IPDPS Workshop on High-Performance, Power-Aware Computing.”

5.2.8 HPC programming models

In the following lectures we will describe the predominant programming models used for developing software for parallel processors and for systems with multiple processors. The main approaches are as follows

- **OpenMP:** stands for the Open Multi-Processing standard. It enables a programmer to write code that can fork multiple threads to handle several tasks at the same time on multi-core processors. The OpenMP API consists of functions to manage the number of threads that are forked, to report wall clock time, and to find a thread number. Preprocessor compiler directives (typically starting with `#pragma omp`) are used to create parallel regions in code where commands are executed by multiple threads.

OpenMP is often used to make an existing code run in parallel by light annotation. OpenMP is predicated on all threads having access to shared memory and this limits it typically to running on laptops, desktops, single compute nodes, and recently graphics processing units. Results from using OpenMP can be mixed with variable performance gains depending on the specific problem, quality of implementation, and threading capability of the target processor(s). We will explore the OpenMP philosophy, API, and programming directives in lectures ?? and ??.

⁵Of note, Dr. Feng is currently a Professor of Computer Science at VT !

You will be able to develop and test OpenMP based codes on your laptop using your VirtualBox after which you can deploy it to a *single* compute node on the Advanced Research Computing clusters.

- **MPI:** stands for the Message Passing Interface. It enables a programmer to write a program that is executed in multiple instances (called processes) at the same time.

There is no requirement that MPI processes must execute on the same computer. If computers on planet earth and out in space on Mars were networked together then the MPI program could run on both at the same time. The important aspect of an MPI program is that every process executing the program can communicate by passing messages to other instance of the MPI program processes.

In our hypothetical case a message sent from earth to Mars would take about 13 minutes according to the [European Space Agency](#). In fact the minimum time it takes to send a message is so fundamental to distributed computing models (as enabled by MPI) that it has a name: latency. Every distributed system has a range of latencies depending on the topology of the network connecting the compute nodes, the type of connections (wire or fibre optic cables), the time lost when messages pass through layers of the operating system, and so on. Usually latency is on the order of milliseconds, with the earth-Mars latency being atypical.

Just like programming in OpenMP, your mileage may vary when using MPI. It is conceptually a little harder to understand what exactly happens when an MPI program executes. However, it is possible to create parallel codes using MPI that scale relatively efficiently up to a million MPI processes running on the largest CPU based supercomputers [3]. We will explore the MPI philosophy, API, and programming directives in lectures ??,??, ??, and ??.

You will be able to develop and test MPI codes on your laptop using your VirtualBox after which you can deploy it to *multiple* compute node on the Advanced Research Computing clusters.

- **CUDA:** stands for Compute Unified Device Architecture. It consists of an API and extensions to C. The API enables a programmer to migrate data to and from a GPU from a host computer and to queue up tasks to run on the GPU. The C extensions in CUDA are used by programmers to write so called “kernels” that are executed by multiple threads on a GPU.

The off-load model of GPU computing obliges the programmer to be aware that the CPU and GPU maintain separate physical memory spaces. Data needs to be copied between these spaces and this incurs latency penalties for GPU tasks. Experience CUDA programmers will try to minimize the amount of data that gets copied between the host computer (simply referred to as the “host”) and the attached GPU (also called the “device”). A typical model is to set up a problem on the host, copy data over to the device, then do as much work as possible on the device before copying the result back to the host.

It is important to note that CUDA is a proprietary API and programming language provided by NVIDIA and is typically restricted to being used on their GPUs. This arrangement is sometimes referred to as a vendor locked ecosystem. Developing software under such an arrangement means that the resulting code depends on continued support of a commercial company to be usable. This obviously entails some risk.

Just like programming in OpenMP and MPI, your mileage may vary when using CUDA. It is conceptually quite a bit harder to understand what exactly happens when an CUDA program executes on a GPU. However, carefully crafted CUDA GPU code can achieve a high percentage of peak performance.

You may be able to use your laptop or desktop to develop and test CUDA code if it has an NVIDIA GPU. However it may not be possible to do this through the VirtualBox as you may not be able to access the GPU through the virtualization layer. You will be able to develop

and test CUDA codes on a GPU equipped compute node on the Cascades Advanced Research Computing cluster.

There are additional GPU programming models that we will introduce but not discuss directly.

- AMD HIP: stands for [Heterogeneous-Compute Interface for Portability](#). It enables a programmer to write code that can execute on AMD GPUs or on NVIDIA GPUs (via translation). It lacks some CUDA functionality that relies on hardware specific features exclusive to NVIDIA GPUs, but otherwise the API calls are very similar except `cuda` is replaced with `hip` in function names.
- OpenCL: stands for [Open Computing Language](#). OpenCL is an open standard proposed by the [Khronos Group](#) and brings a CUDA inspired programming model to CPUs, GPUs, field programmable gate arrays (FPGAs), and even embedded processors. The start up cost for programmers starting with vanilla OpenCL is higher than CUDA. The extra complexity of using OpenCL boils down to the broad range of hardware processors and OpenCL implementations that have to be accommodated when starting an OpenCL computation. One notable feature of using OpenCL is that unlike CUDA it does not require a special compiler as it is entirely library based.
- SyCL: [single source heterogeneous programming for OpenCL](#) brings a C++ front end to OpenCL.
- OpenACC: stands for [Open Accelerator](#). It is similar in spirit to OpenMP but designed to target primarily offload computing with GPUs.

5.2.9 That’s all “nice” but what about the cloud ?

Where HPC goes, the ☁ follows. The mechanics and traditions of Linux terminal based computing that originated in large part with HPC translate immediately to the cloud. The “cloud” is after all largely just a collection of compute and storage nodes likely running Linux. Cloud computing instances come equipped with every available CPU and some include GPU(s).

The heavy computational demands of training deep neural networks has resulted in the major cloud providers making compute instances available with heavyweight GPUs, in particular the NVIDIA Volta V100 (or Ampere A100) GPU is a popular configuration option because of its built-in specialized tensor cores.

The price of an instance depends on the capability of the CPU/GPU. To maximize the return on investment you really should use every CPU/GPU core available and that obliges you to use and/or write computer codes that are enabled for parallel computing!

5.2.10 Definitions

Here are some acronyms and phrases that you will hear repeatedly in the rest of the semester.

- [Single Instruction Multiple Data](#) (SIMD): one instruction stream (i.e. set of code instructions) being used to simultaneously process multiple sets of data in lockstep.

- **Single Program Multiple Data** (SPMD): one program executed multiple times on separate data sets, not forced to process in lockstep. MPI programs fall loosely into this category.
- **Flynn's taxonomy**: a classification scheme for different parallel programming models.
- **Compiler directive** (or pragma): an instruction in source code that a compiler may elect to modify the way the program is compiled.
- **Graphics Processing Unit** (GPU, device, or accelerator): a quasi-autonomous computer attached to a host computer.
- **Vector processing unit** (or SIMD unit): hardware capable of executing a SIMD program.
- **Vector lane**: a single floating point unit in a vector processing unit.
- **Thread**: a single stream of program instructions.
- **Warp** (or wavefront): a group of threads that are processed in SIMD fashion, i.e. in lockstep, concurrently on a GPU SIMD unit.
- **Single Instruction Multiple Threads** (SIMT): NVIDIA refers to their specific GPU version of SIMD as SIMT, distinct from SIMD, because each lane in the vector processing units of their GPU cores can execute either an instruction or a non-operation (NOP).
- **Network Interface Controller** (NIC): hardware that sits between the CPU and the external network.
- **CPU Register**: fast memory store collocated with processor core.

5.3 Performance

Of course our main purpose in writing parallel programs and using HPV systems is usually increased performance. So what can we expect? And how can we evaluate our programs?

5.3.1 Speedup and Efficiency

Usually the best we can hope to do is to equally divide the work among the cores, while at the same time introducing no additional work for the cores. If we succeed in doing this, and we run our program with p cores, one thread or process on each core, then our parallel program will run p times faster than the serial program. If we call the serial run-time T_{serial} and our parallel run-time T_{parallel} , then the best we can hope for is $T_{\text{parallel}} = T_{\text{serial}} / p$. When this happens, we say that our parallel program has linear speedup.

In practice, we're unlikely to get linear speedup because the use of multiple processes/threads almost invariably introduces some overhead. For example, shared memory programs will almost always have critical sections, which will require that we use some mutual exclusion mechanism such as a mutex. The calls to the mutex functions are overhead that's not present in the serial program, and the use of the mutex forces the parallel program to serialize execution of the critical section. Distributed-memory programs will almost always need to transmit data across the network, which is usually much slower than local memory access. Serial programs, on the other hand, won't have these overheads. Thus, it will be very unusual for us to find that our parallel programs get linear speedup. Furthermore, it's

likely that the overheads will increase as we increase the number of processes or threads, that is, more threads will probably mean more threads need to access a critical section. More processes will probably mean more data needs to be transmitted across the network.

So if we define the speedup of a parallel program to be

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

then linear speedup has $S = p$, which is unusual. Furthermore, as p increases, we expect S to become a smaller and smaller fraction of the ideal, linear speedup p . Another way of saying this is that S/p will probably get smaller and smaller as p increases. Table 2.4 shows an example of the changes in S and S/p as p increases.¹ This value, S/p , is sometimes called the efficiency of the parallel program. If we substitute the formula for S , we see that the efficiency is

$$E = \frac{S}{p} = \frac{\left(\frac{T_{\text{serial}}}{T_{\text{parallel}}}\right)}{p} = \frac{T_{\text{serial}}}{p \cdot T_{\text{parallel}}}$$

p	1	2	4	8	16
S	1.0	1.9	3.6	6.5	10.8
$E = S/p$	1.0	0.95	0.90	0.81	0.68

Table 5.2: Speedups and Efficiencies of a Parallel Program

It's clear that T_{parallel} , S , and E depend on p , the number of processes or threads. We also need to keep in mind that T_{parallel} , S , E , and T_{serial} all depend on the problem size. For example, if we halve and double the problem size of the program whose speedups are shown in Table 5.2, we get the speedups and efficiencies shown in Table 5.3. We see that in this example, when we increase the problem size,

	p	1	2	4	8	16
Half	S	1.0	1.9	3.1	4.8	6.2
	E	1.0	0.95	0.78	0.60	0.39
Original	S	1.0	1.9	3.6	6.5	10.8
	E	1.0	0.95	0.90	0.81	0.68
Double	S	1.0	1.9	3.9	7.5	14.2
	E	1.0	0.95	0.98	0.94	0.89

Table 5.3: Speedups and Efficiencies of a Parallel Program on Different Problem Sizes

the speedups and the efficiencies increase, while they decrease when we decrease the problem size. This behavior is quite common. Many parallel programs are developed by dividing the work of the serial program among the processes/threads and adding in the necessary "parallel overhead" such as mutual exclusion or communication. Therefore, if T_{overhead} denotes this parallel overhead, it's often the case that

$$T_{\text{parallel}} = \frac{T_{\text{serial}}}{p} + T_{\text{overhead}}.$$

Furthermore, as the problem size is increased, T_{overhead} often grows more slowly than T_{serial} . When this is the case the speedup and the efficiency will increase. This is what your intuition should tell you: there's more work for the processes/threads to do, so the relative amount of time spent coordinating the work of the processes/threads should be less.

A final issue to consider is what values of T_{serial} should be used when reporting speedups and efficiencies. Some authors say that T_{serial} should be the run-time of the fastest program on the fastest processor available. In practice, most authors use a serial program on which the parallel program was based and run it on a single processor of the parallel system. So if we were studying the performance of a parallel shell sort program, authors in the first group might use a serial radix sort or quicksort on a single core of the fastest system available, while authors in the second group would use a serial shell sort on a single processor of the parallel system. We'll generally use the second approach.

5.3.2 Amdahl's law

In this section we consider an abstract model for a fixed task that is made parallel using P processors. This model was first discussed by [Gene Amdahl](#) at a computing conference in 1967 with a sketched discussion in proceedings [4]⁶. According to scholar.google.com there are now over 5000 articles that reference Amdahl's law despite the original description lacking a formal prescription for the "law".

The main idea of the model is to assume that for every calculation there is a part of the task that cannot benefit from using more processors, which we will refer to as the "serial fraction" of the task denoted by $s \in [0, 1]$. The model further assumes that the remainder of the task, or the "parallel fraction", can be performed with perfect parallel efficiency.

In Amdahl's model the time that P processors take to complete the task is assumed to be

$$T_P = sT_1 + \frac{1-s}{P}T_1.$$

In words the time it takes P processes to complete the tasks consists of the time it takes one processor to complete the serial part of the task plus the time it takes P to complete the remaining part of the task that is perfectly parallel.

The parallel speed up for this model is thus

$$S_P = \frac{T_1}{T_P} = \frac{T_1}{sT_1 + \frac{1-s}{P}T_1} = \frac{1}{s + \frac{1-s}{P}}.$$

Note: Amdahl model speed up is relative and independent T_1 .

There are some interesting limits for the Amdahl speed up:

- $s = 0$: in the case of a perfectly parallelizable task with no serial bottleneck the speed up is $S_P = P$.
- $s = 1$: in the case of a purely serial task the speed up is $S_P = 1$.
- $s = 0.5$: when half the task is serial then the speed up approaches 2 from below as $P \rightarrow \infty$. Plainly speaking that you can throw as many processors at the task as you like, on the largest computer in the world, and you only get a factor of 2 speed up!
- $s = 0.1$: if only ten percent of the task is bound to be serial then the maximum speed up you can get is ten!
- $s = 0.01$: if only one percent of the task is serial you can still only get up to 100 times speed up. It doesn't matter if you use a computer with a thousand processors or even a million processors your code can only run a hundred times faster.

⁶Assignment: read and digest the paper by Amdahl

In Figure 5.6 we plot the speed up curves for a range of different serial fractions. It should be striking that if only one percent of your computational task is bound to be serial then the best you can expect is a hundred times speed up. With that in mind you might wonder why equip the [Summit supercomputer](#) with over 27,000 GPUs that each in turn has 2560 floating point units ? That system can process a task with in excess of 70 million processing elements. By Amdahl's law we would require that a tiny fraction of the task be serial to fully utilize the Summit system.

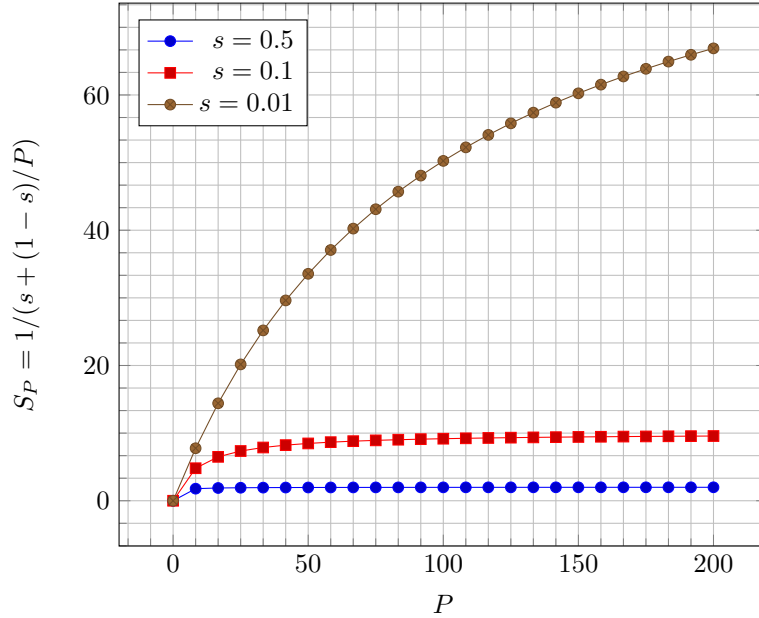


Figure 5.6: Amdahl speed up S_P for a range of serial fraction values.

In Figure 5.7 we illustrate the dependence of the expected speed up for 2560 processors working for a fixed size problem as function of serial fraction. It is apparent that the parallel speed up drops off very quickly with even modest serial fraction.

The fundamental issue here is that for a fixed size problems increasing P drives the computational time for the parallel part of the problem to zero but does not change the time to compute the serial part. In the next section we introduce a less pessimistic model for parallel computing than Amdahl's parallel model.

5.3.3 Gustafson's law

In 1988 John Gustafson revisited the doomsaying aspect of Amdahl's law [5] for large scale parallel computing with many processors. Gustafson tweaked Amdahl's model by assuming that

1. The size of the problem grows in proportion to the number of processors (solve larger problems on larger systems).
2. The serial fraction of the problem does not grow with increasing number of processors.

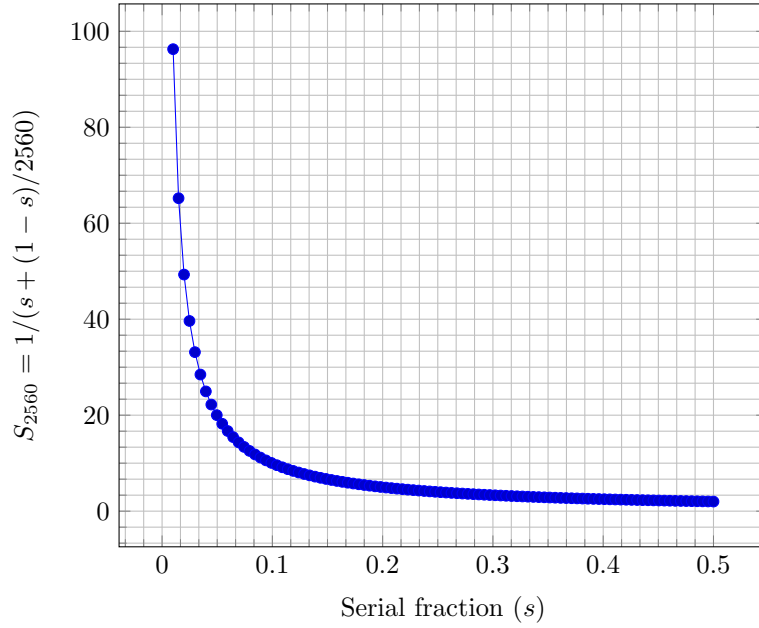


Figure 5.7: Amdahl speed up for $P = 2650$ processors for a range of serial fraction values.

In Gustafson's model the work performed by P processors is

$$W_P = sW_0 + P(1-s)W_0,$$

which represents a fixed amount of serial work in the first expression and P times the amount of the baseline parallelizable work W_0 for one processor.

The model parallel speed up will be the time it takes one processor to complete W_P work divided by the time it takes P processors to complete the work. Assuming that all processors complete work at the same rate, the speed up in this case is given by

$$S_P := \frac{sW_0 + P(1-s)W_0}{sW_0 + \frac{P(1-s)W_0}{P}}.$$

Simplifying this we get

$$S_P = s + (1-s)P.$$

Thus we can expect the parallel speed up as defined above to be linear in the number of processors P but with slope dictated by the parallel fraction. We only lose sP in speed up relative to an optimal speed up. In Figure 5.8 we illustrate the speed up estimate for a range of P and serial fractions s .

Gustafson's law is much less pessimistic than Amdahl's law. Following the logic we can better utilize a HPC system by increasing the problem size while also minimizing the serial component of the task to increase the slope of the speed up chart. The flip side of this analysis is that to fully occupy the 27,000+ GPUs on Summit we will need to solve problems that are 27,000 times larger than a single GPU can solve.

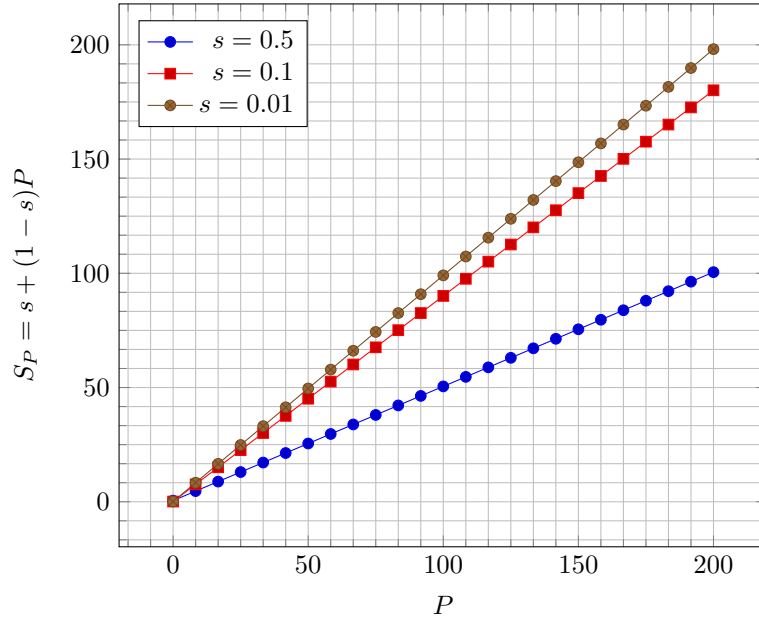


Figure 5.8: Gustafson speed up S_P for a range of serial fraction values, assuming the parallel component of the computational task grows linearly with the processor count.

5.3.4 Scalability

In discussions of parallel program performance, scalability has a somewhat formal definition. Suppose we run a parallel program with a fixed number of processes/threads and a fixed input size, and we obtain an efficiency E . Suppose we now increase the number of processes/threads that are used by the program. If we can find a corresponding rate of increase in the problem size so that the program always has efficiency E , then the program is scalable.

As an example, suppose that $T_1 = n$, where the units of T_1 are in microseconds, and n is also the problem size. Also suppose that $T_P = n/p + 1$. Then

$$E = \frac{n}{p(n/p + 1)} = \frac{n}{n + p}$$

To see if the program is scalable, we increase the number of processes/threads by a factor of k , and we want to find the factor x that we need to increase the problem size by so that E is unchanged. The number of processes/threads will be kp and the problem size will be xn , and we want to solve the following equation for x :

$$E = \frac{n}{n + p} = \frac{xn}{xn + kp}$$

Well, if $x = k$, there will be a common factor of k in the denominator $xn + kp = kn + kp = k(n + p)$, and we can reduce the fraction to get

$$\frac{xn}{xn + kp} = \frac{kn}{k(n + p)} = \frac{n}{n + p}$$

In other words, if we increase the problem size at the same rate that we increase the number of processes/threads, then the efficiency will be unchanged, and our program is scalable.

There are a couple of cases that have special names. If when we increase the number of processes/threads, we can keep the efficiency fixed without increasing the problem size, the program is said to be **strongly** scalable. If we can keep the efficiency fixed by increasing the problem size at the same rate as we increase the number of processes/threads, then the program is said to be **weakly** scalable. The program in our example would be weakly scalable.